BIOT 6900 · AI-POWERED BIOTECH WORKFLOWS

Module 1 — Lab Guide

Computational Environment Setup & Tool Ecosystem

Week 1 · September 10, 2026 Instructor: Prof. Sudhakaran Prabakaran, FRSB
Thursday, 6:00–8:30 pm · 227 Richards Hall · s.prabakaran@northeastern.edu

This guide walks you through everything you need to do for Module 1, in class and immediately after. Work through each part in order; the whole environment builds on itself. Everything you produce this week goes into your course GitHub repository.

Contents

Contents 1
1. **Overview & Learning Objectives** 2
2. **Before You Begin** 2
3. **Part A — Environment Setup** 3
4. **Part B — BioPython Quickstart** 5
5. **Part C — Biomedical Database API Exploration** 5
6. **Part D — Download Sample Multi-Omics Datasets** 7
7. **Deliverable & Submission** 7
8. **Grading Rubric** 7
9. **Troubleshooting & Support** 8

1. Overview & Learning Objectives

Module 1 sets up the computational workbench you'll use for the rest of the course and gives you your first hands-on contact with real biological data. By the end of this module you will be able to:

- **Stand up a reproducible Python environment** using Anaconda, conda, and Jupyter.
- **Manage your work with Git and GitHub** in a documented course repository.
- **Query four core biomedical databases** — PubMed, UniProt, PDB, and the GWAS Catalog — programmatically via their APIs.
- **Retrieve and load multi-omics datasets** in preparation for Week 2's target-identification work.

Course outcomes touched this week: LO1 (apply industry-standard tools) and LO5 (communicate workflows through professional documentation).

> **Platform for this week:** Do Week 1 locally in your biot6900 environment — building it yourself is the point, and it's graded. A Colab fallback notebook and the full week-by-week platform guide are posted on Canvas.

2. Before You Begin

System requirements

- Personal laptop (macOS, Windows, or Linux) with at least 8 GB RAM — 16 GB recommended.
- At least 50 GB of free storage and a reliable internet connection.
- Administrator rights to install software.

Accounts to create (free)

- **GitHub** — sign up at github.com if you don't already have an account.
- **Google account** — for Google Colab, used later in the course for GPU-heavy work.

> **Heads-up:** Several APIs ask you to identify yourself by email (NCBI Entrez in particular). Use your Northeastern address so requests aren't rate-limited or blocked.

3. Part A — Environment Setup

Do these steps once. The goal is a single, isolated environment named biot6900 that every notebook this term will run in.

A1. Install Anaconda

1. Download the Anaconda Distribution installer for your operating system from anaconda.com.
2. Run the installer and accept the defaults. This installs Python 3, the conda package manager, and Jupyter.
3. Open a terminal (macOS/Linux) or the Anaconda Prompt (Windows) and confirm conda is available:

```
conda --version
```

A2. Create the course environment

Create one isolated environment and install the packages you'll need this week:

```
conda create -n biot6900 python=3.11 -y
conda activate biot6900
pip install biopython pandas numpy requests jupyter
```

You'll know it worked when your prompt shows **(biot6900)** at the start of the line. Activate this environment every time you work on the course.

A3. Launch Jupyter and verify Python

```
jupyter notebook
# in a new notebook cell:
import sys
print(sys.version)
```

Jupyter opens in your browser. Create a new notebook, run the cell above, and confirm you're on Python 3.11.

A4. Install and configure Git; connect GitHub

1. Install Git (git-scm.com) if it isn't already on your machine, then check the version and set your identity:

```
git --version
git config --global user.name "Your Name"
git config --global user.email "your.email@northeastern.edu"
```

2. On GitHub, create a new repository named **biot6900**. Then clone it locally, add a file, and make your first commit:

```
git clone https://github.com/<your-username>/biot6900.git
```

```
cd biot6900
echo "# BIOT 6900 coursework" > README.md
git add README.md
git commit -m "Initial commit"
git push
```

A5. Google Colab check

Open colab.research.google.com, sign in with your Google account, and create a new notebook. Run a single cell (for example print("Colab works")) to confirm access. We won't use Colab heavily yet, but confirming access now saves time later.

4. Part B — BioPython Quickstart

BioPython is the workhorse library for biological computation in Python. You installed it in Part A; confirm it's importable in your course environment:

```
import Bio
print(Bio.__version__)
```

A quick sanity check that also previews what's ahead — parsing a short sequence:

```
from Bio.Seq import Seq
dna = Seq("ATGGCCATTGTAATGGGCCGCTGA")
print("Length:", len(dna))
print("Protein:", dna.translate())
```

5. Part C — Biomedical Database API Exploration

The core skill this week: reaching real data programmatically. Each database below follows the same rhythm — build a query, call the API, parse the response. Run each snippet, then record one result and a one-line note on what you retrieved.

C1. PubMed — via Bio.Entrez

NCBI's Entrez interface (wrapped by BioPython) gives you programmatic access to PubMed literature.

```
from Bio import Entrez
Entrez.email = "your.email@northeastern.edu"   # required by NCBI
handle = Entrez.esearch(db="pubmed", term="AlphaFold protein structure", retmax=5)
record = Entrez.read(handle)
print("PMIDs:", record["IdList"])
```

Record: the list of PubMed IDs returned. Note what search term you used.

C2. UniProt — via the REST API

UniProt holds protein sequence and functional annotation. Its REST API returns clean JSON. Here we fetch the human tumor-suppressor p53 (accession P04637):

```
import requests
r = requests.get("https://rest.uniprot.org/uniprotkb/P04637.json")
data = r.json()
name = data["proteinDescription"]["recommendedName"]["fullName"]["value"]
print("Protein:", name)
```

Record: the protein name returned, and the accession you queried.

C3. PDB — via the RCSB REST API

The Protein Data Bank stores experimentally determined 3D structures. Query metadata for a p53 structure (PDB ID 1TUP), then note where the coordinate file lives:

```
import requests
r = requests.get("https://data.rcsb.org/rest/v1/core/entry/1TUP")
print("Title:", r.json()["struct"]["title"])
# the structure file itself:
# https://files.rcsb.org/download/1TUP.pdb
```

Record: the structure title, and the PDB ID you queried.

C4. GWAS Catalog — via the EBI REST API

The GWAS Catalog links genetic variants to traits. Look up a well-known variant (rs7412, in APOE):

```
import requests
base = "https://www.ebi.ac.uk/gwas/rest/api"
r = requests.get(f"{base}/singleNucleotidePolymorphisms/rs7412")
data = r.json()
print("Variant:", data.get("rsId"))
```

Record: the variant ID returned, and one association you find interesting from the response.

> **If a request fails:** Check your internet connection first, then confirm the URL exactly. Public APIs occasionally rate-limit — wait a moment and retry, or add your NCBI/EBI email where requested.

6. Part D — Download Sample Multi-Omics Datasets

Next week we integrate transcriptomics, proteomics, and genomics. Tonight you'll stage one small sample from the public repositories those layers come from. Specific accessions and download links are posted on Canvas; the loading pattern is the same regardless of which you're assigned.

- **GEO (Gene Expression Omnibus)** — transcriptomics. Download the series matrix for the assigned GSE accession.
- **GTEx** — expression across human tissues. Download the assigned gene-level subset from the GTEx portal.
- **cBioPortal** — multi-omics cancer data with clinical annotation. Export the assigned study slice.

Load each file, confirm it opens, and print its dimensions so you know the data arrived intact:

```
import pandas as pd
df = pd.read_csv("data/<assigned_file>.csv")   # adjust path / separator as needed
print("Shape:", df.shape)
df.head()
```

Record: the shape (rows × columns) of each dataset and one sentence on what it contains.

7. Deliverable & Submission

Bundle your work into a single Jupyter notebook plus a short README, committed to your course repository.

Start from the provided starter notebook **(module1_setup.ipynb)** on Canvas — it already contains the scaffold and TODO cells for Parts A–D. Fill them in as you work through this guide.

1. **Environment verification** — a notebook cell (or short section) printing your Python, conda, Git, and BioPython versions and confirming the biot6900 environment is active.
2. **Database API exploration** — the four queries from Part C, each with the result and your one-line note.
3. **Sample datasets** — the load-and-shape checks from Part D.

Submit by: committing **module1_setup.ipynb** and a README to your **biot6900** GitHub repository, then posting the repository link on Canvas.

> **Late policy (per syllabus):** −10% up to 1 day late, −20% up to 2 days, −50% up to 3 days, no credit after the 3rd day unless an alternate schedule has been discussed in advance.

8. Grading Rubric

This assignment follows the course 60 / 40 philosophy: **60% computational execution** and **40% biological interpretation and communication.**

Component	Weight	What we look for
Environment verified & runs *(execution)*	30%	Notebook runs top-to-bottom with no errors; correct Python, conda, Git, and BioPython versions printed; course environment active.
Database queries executed *(execution)*	30%	All four APIs called successfully; at least one result retrieved from each; sound, appropriate methodology.
Interpretation & notes *(interpretation)*	25%	Short, clear notes on what each query returned and why that data type matters for target discovery.
Documentation & repo hygiene *(communication)*	15%	README present; clean commit; readable, well-labeled notebook; results reproducible by the grader.

Partial credit is awarded for a correct conceptual approach even with minor technical errors — explaining what you were attempting matters.

9. Troubleshooting & Support

Common snags

- **"conda: command not found"** — reopen your terminal after installing Anaconda, or use the Anaconda Prompt on Windows.
- **Wrong Python / packages missing** — confirm your prompt shows (biot6900); if not, run conda activate biot6900.
- **Entrez request rejected** — make sure you set Entrez.email before calling any Entrez function.
- **SSL or connection errors on an API** — check your network, then retry; corporate/campus firewalls occasionally block direct calls.

Where to get help

- **Weekly computational office hours** — two dedicated hours for setup and coding questions.
- **Code templates** — a scaffolded notebook accompanies this guide on Canvas.
- **Optional pre-course Python module** — a self-paced refresher if you'd like to shore up the basics.
- **Email** — include your name and class title; allow up to 48 hours for a reply.